

Assignment 2 - FMLA

Deepika

2026-10-03

```
# Load required packages
```

```
library(class)
```

```
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```
library(dplyr)
```

```
##
```

```
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
## filter, lag
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
## intersect, setdiff, setequal, union
```

```
#Load the dataset
```

```
getwd()
```

```
## [1] "C:/Users/Manager/OneDrive/Documents/FUNDAMENTALS OF MACHINE LEARNING"
```

```
list.files()
```

```
## [1] "Assignment-2.html"
```

```
## [2] "Assignment-2.pdf"
```

```
## [3] "Assignment-2.Rmd"
```

```
## [4] "Assignment 2.Rmd"
```

```
## [5] "Assignment and Discussion Topic"
```

```
## [6] "Classifications Slides and Transcript.pdf"
```

```
## [7] "diabetes_prediction_dataset.csv"
```

```
## [8] "FMLA-Assignment-1--12-sep-2026.pdf"
```

```
## [9] "FMLA Assignment 1- 12 sep 2026.Rmd"
```

```
## [10] "Key Models Slides and Transcript.pdf"
## [11] "Machine Learning Types Slides and Transcript.pdf"
## [12] "ML Discussion .docx"
## [13] "Module 1 Introduction Slides and Transcript.pdf"
## [14] "Module 3 Discussion Instructions.pdf"
## [15] "Module 3 Introduction Slides and Transcript.pdf"
```

```
file.exists("UniversalBank.csv")
```

```
## [1] FALSE
```

```
getwd()
```

```
## [1] "C:/Users/Manager/OneDrive/Documents/FUNDAMENTALS OF MACHINE LEARNING"
```

```
file.exists("UniversalBank.csv")
```

```
## [1] FALSE
```

```
bank <- read.csv("C:/Users/Manager/OneDrive/Documents/DataSets/UniversalBank.csv")
```

```
head(bank)
```

```
##   ID Age Experience Income ZIP.Code Family CCAvg Education Mortgage
## 1  1  25         1     49   91107      4   1.6         1         0
## 2  2  45        19     34   90089      3   1.5         1         0
## 3  3  39        15     11   94720      1   1.0         1         0
## 4  4  35         9    100   94112      1   2.7         2         0
## 5  5  35         8     45   91330      4   1.0         2         0
## 6  6  37        13     29   92121      4   0.4         2        155
##   Personal.Loan Securities.Account CD.Account Online CreditCard
## 1             0                 1           0         0         0
## 2             0                 1           0         0         0
## 3             0                 0           0         0         0
## 4             0                 0           0         0         0
## 5             0                 0           0         0         1
## 6             0                 0           0         1         0
```

```
dim(bank)
```

```
## [1] 5000   14
```

```
names(bank)
```

```
## [1] "ID"           "Age"          "Experience"
## [4] "Income"       "ZIP.Code"     "Family"
## [7] "CCAvg"        "Education"    "Mortgage"
## [10] "Personal.Loan" "Securities.Account" "CD.Account"
## [13] "Online"       "CreditCard"
```

```
str(bank)
```

```
## 'data.frame':    5000 obs. of  14 variables:
## $ ID           : int  1 2 3 4 5 6 7 8 9 10 ...
## $ Age          : int  25 45 39 35 35 37 53 50 35 34 ...
## $ Experience    : int  1 19 15 9 8 13 27 24 10 9 ...
## $ Income       : int  49 34 11 100 45 29 72 22 81 180 ...
## $ ZIP.Code     : int  91107 90089 94720 94112 91330 92121 91711 93943 90089 93023 ...
## $ Family       : int  4 3 1 1 4 4 2 1 3 1 ...
## $ CCAvg        : num  1.6 1.5 1 2.7 1 0.4 1.5 0.3 0.6 8.9 ...
## $ Education    : int  1 1 1 2 2 2 2 3 2 3 ...
## $ Mortgage     : int  0 0 0 0 0 155 0 0 104 0 ...
## $ Personal.Loan : int  0 0 0 0 0 0 0 0 0 1 ...
## $ Securities.Account: int  1 1 0 0 0 0 0 0 0 0 ...
## $ CD.Account   : int  0 0 0 0 0 0 0 0 0 0 ...
## $ Online       : int  0 0 0 0 0 1 1 0 1 0 ...
## $ CreditCard   : int  0 0 0 0 1 0 0 1 0 0 ...
```

```
table(bank$Personal.Loan)
```

```
##
##    0    1
## 4520  480
```

```
#Create Dummy Variables
```

```
bank$Education <- factor(bank$Education)
education_dummies <- dummyVars(~ Education, data = bank)

education_data <- predict(education_dummies, newdata = bank)
education_data <- as.data.frame(education_data)

head(education_data)
```

```
##   Education.1 Education.2 Education.3
## 1           1           0           0
## 2           1           0           0
## 3           1           0           0
## 4           0           1           0
## 5           0           1           0
## 6           0           1           0
```

```
names(education_data) <- c("Education_1", "Education_2", "Education_3")
```

```
#Create the predictor dataset
```

```
X <- bank %>%
  select(
    Age,
    Experience,
    Income,
```

```

    Family,
    CCAvg,
    Mortgage,
    Securities.Account,
    CD.Account,
    Online,
    CreditCard
  )

#Adding dummy variable

X <- cbind(X, education_data)
head(X)

##   Age Experience Income Family CCAvg Mortgage Securities.Account CD.Account
## 1  25          1     49      4   1.6         0              1          0
## 2  45         19     34      3   1.5         0              1          0
## 3  39         15     11      1   1.0         0              0          0
## 4  35          9    100      1   2.7         0              0          0
## 5  35          8     45      4   1.0         0              0          0
## 6  37         13     29      4   0.4        155              0          0
##   Online CreditCard Education_1 Education_2 Education_3
## 1      0          0             1           0           0
## 2      0          0             1           0           0
## 3      0          0             1           0           0
## 4      0          0             0           1           0
## 5      0          1             0           1           0
## 6      1          0             0           1           0

#Create the target variable

y <- factor(bank$Personal.Loan, levels = c(0, 1))
table(y)

## y
##   0    1
## 4520 480

#Split the dataset

set.seed(123)

#Create the partition
train_index <- createDataPartition(
  y,
  p = 0.60,
  list = FALSE
)

#Create validation and validation dataset

```

```
X_train <- X[train_index, ]
X_valid <- X[-train_index, ]
```

```
y_train <- y[train_index]
y_valid <- y[-train_index]
```

```
#Check the dataset
```

```
nrow(X_train)
```

```
## [1] 3000
```

```
nrow(X_valid)
```

```
## [1] 2000
```

```
#Standardize the Predictors
```

```
preprocess <- preProcess(
  X_train,
  method = c("center", "scale")
)
X_train_scaled <- predict(preprocess, X_train)
X_valid_scaled <- predict(preprocess, X_valid)
```

```
#Question 1
```

```
#Perform a k-NN classification with all predictors except ID and ZIP code using k = 1.
```

```
knn_1 <- knn(
  train = X_train_scaled,
  test = X_valid_scaled,
  cl = y_train,
  k = 1
)
table(knn_1)
```

```
## knn_1
##      0      1
## 1849  151
```

```
#Create the Confusion Matrix
```

```
confusionMatrix(
  knn_1,
  y_valid,
  positive = "1"
)
```

```
## Confusion Matrix and Statistics
```

```
##
##           Reference
## Prediction    0    1
##           0 1793   56
##           1   15  136
##
##           Accuracy : 0.9645
##           95% CI : (0.9554, 0.9722)
##           No Information Rate : 0.904
##           P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.7739
##
## Mcnemar's Test P-Value : 2.063e-06
##
##           Sensitivity : 0.7083
##           Specificity : 0.9917
##           Pos Pred Value : 0.9007
##           Neg Pred Value : 0.9697
##           Prevalence : 0.0960
##           Detection Rate : 0.0680
##           Detection Prevalence : 0.0755
##           Balanced Accuracy : 0.8500
##
##           'Positive' Class : 1
##
```

```
customer <- data.frame(
  Age = 40,
  Experience = 10,
  Income = 84,
  Family = 2,
  CCAvg = 2,
  Mortgage = 0,
  Securities.Account = 0,
  CD.Account = 0,
  Online = 1,
  CreditCard = 1,
  Education_1 = 0,
  Education_2 = 1,
  Education_3 = 0
)
customer
```

```
##   Age Experience Income Family CCAvg Mortgage Securities.Account CD.Account
## 1  40         10     84      2      2         0              0          0
##   Online CreditCard Education_1 Education_2 Education_3
## 1      1          1           0           1           0
```

#Scaling

```
X_train_scaled <- scale(X_train)
```

#Scale the validation data using the training mean and standard deviation

```
customer_scaled <- scale(
  customer,
  center = attr(X_train_scaled, "scaled:center"),
  scale = attr(X_train_scaled, "scaled:scale")
)

#Run knn with k=1
customer_k1 <- knn(
  train = X_train_scaled,
  test = customer_scaled,
  cl = y_train,
  k = 1
)

customer_k1
```

```
## [1] 0
## Levels: 0 1
```

Here k-NN with $k = 1$, so the customer is classified as 0 (Personal Loan = No). Therefore, the model predicts that this customer will not accept the personal loan offer.

#Question 2

#What is a choice of k that balances between overfitting and ignoring the predictor information?

```
k_values <- seq(1, 25, 2)
accuracy <- c()

for (k in k_values) {
  knn_model <- knn(
    X_train_scaled,
    X_valid_scaled,
    y_train,
    k = k
  )

  accuracy <- c(
    accuracy,
    mean(knn_model == y_valid)
  )
}

results <- data.frame(
  k = k_values,
  accuracy = accuracy
)

results
```

```
##      k accuracy
## 1    1  0.9645
## 2    3  0.9635
## 3    5  0.9595
## 4    7  0.9585
## 5    9  0.9535
## 6   11  0.9515
## 7   13  0.9495
## 8   15  0.9470
## 9   17  0.9445
## 10  19  0.9440
## 11  21  0.9440
## 12  23  0.9435
## 13  25  0.9440
```

```
best_k <- results$k[which.max(results$accuracy)]
best_k
```

```
## [1] 1
```

```
results
```

```
##      k accuracy
## 1    1  0.9645
## 2    3  0.9635
## 3    5  0.9595
## 4    7  0.9585
## 5    9  0.9535
## 6   11  0.9515
## 7   13  0.9495
## 8   15  0.9470
## 9   17  0.9445
## 10  19  0.9440
## 11  21  0.9440
## 12  23  0.9435
## 13  25  0.9440
```

$k = 1$ because it produced the highest validation accuracy of 96.45% among the k values tested. However, $k = 1$ can be sensitive to individual observations and may overfit the training data.

#Question 3 #Show the confusion matrix for the validation data that results from using the best k .

```
best_prediction <- knn(
  X_train_scaled,
  X_valid_scaled,
  y_train,
  k = best_k
)
table(
```

```

Predicted = best_prediction,
Actual = y_valid
)

```

```

##           Actual
## Predicted    0    1
##           0 1793   56
##           1   15  136

```

#Calculate the Accuracy

```

(1793 + 136) / (1793 + 56 + 15 + 136)

```

```

## [1] 0.9645

```

#The validation confusion matrix shows 1,793 correctly classified customers in class 0 and 136 correctly classified customers in class. The validation accuracy was 96.45%. The sensitivity for the loan-acceptance class was 70.83%, while specificity was 99.17%.

#Question 4 # Classify the customer using the best k

```

customer_best <- knn(
  X_train_scaled,
  customer_scaled,
  y_train,
  k = best_k
)

customer_best

```

```

## [1] 0
## Levels: 0 1

```

#The selected value of k = 1, the customer is classified as 0 (Personal Loan = No).So,the model predicts that the customer will not accept the personal loan.

#Question 5 #Compare the confusion matrix of the test set with that of the training and validation sets. Comment on the differences and their reason.

```

set.seed(123)

train_index <- createDataPartition(
  y,
  p = 0.50,
  list = FALSE
)

```

```

X_train2 <- X[train_index, ]
y_train2 <- y[train_index]

X_remaining <- X[-train_index, ]
y_remaining <- y[-train_index]

```

```
#Split the data
```

```
valid_index <- createDataPartition(  
  y_remaining,  
  p = 0.60,  
  list = FALSE  
)
```

```
#Create Validation and testing
```

```
X_valid2 <- X_remaining[valid_index, ]  
y_valid2 <- y_remaining[valid_index]  
  
X_test2 <- X_remaining[-valid_index, ]  
y_test2 <- y_remaining[-valid_index]  
  
nrow(X_train2)
```

```
## [1] 2500
```

```
nrow(X_valid2)
```

```
## [1] 1500
```

```
nrow(X_test2)
```

```
## [1] 1000
```

```
#Scaling
```

```
X_train2_scaled <- scale(X_train2)  
  
X_valid2_scaled <- scale(  
  X_valid2,  
  center = attr(X_train2_scaled, "scaled:center"),  
  scale = attr(X_train2_scaled, "scaled:scale")  
)  
  
X_test2_scaled <- scale(  
  X_test2,  
  center = attr(X_train2_scaled, "scaled:center"),  
  scale = attr(X_train2_scaled, "scaled:scale")  
)
```

```
#Best k
```

```
pred_train <- knn(  
  X_train2_scaled,  
  X_train2_scaled,  
  y_train2,  
  k = best_k
```

```
)

pred_valid <- knn(
  X_train2_scaled,
  X_valid2_scaled,
  y_train2,
  k = best_k
)

pred_test <- knn(
  X_train2_scaled,
  X_test2_scaled,
  y_train2,
  k = best_k
)
```

#Confusion Matrix

```
#Training
table(
  Predicted = pred_train,
  Actual = y_train2
)
```

```
##           Actual
## Predicted    0    1
##           0 2260    0
##           1    0  240
```

#Validation

```
table(
  Predicted = pred_valid,
  Actual = y_valid2
)
```

```
##           Actual
## Predicted    0    1
##           0 1339   33
##           1   17  111
```

#Test

```
table(
  Predicted = pred_test,
  Actual = y_test2
)
```

```
##           Actual
## Predicted    0    1
##           0  899   28
##           1    5   68
```

#Training: 100% accuracy #Validation: 96.67% accuracy #Test: 96.70% accuracy.

#Difference is overfitting caused by $k = 1$ and the training performance is higher than the validation and test performance because k-NN with $k = 1$ can overfit the training data.